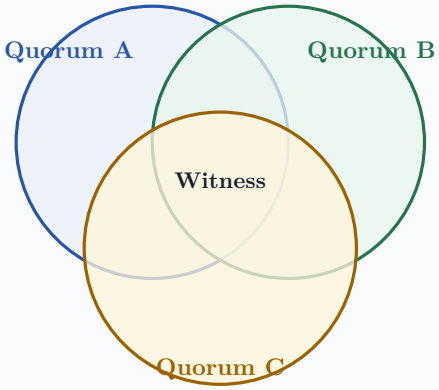
Part Time Parliament

A Literate Guide to Paxos in Zig



Paxos protocol design, optimization, and implementation

Version 0.1.0

Paxos Zig contributors

About this book

This book explains one algorithm. It also explains one implementation of that algorithm. The two tasks are kept together. A line of code is easier to trust when we know the fact that requires it. A proof is easier to remember when we can point to the field that carries its meaning.

The source is written in Typst. The diagrams use Fletcher and CeTZ. The code is Zig 0.16. The protocol is Paxos as presented by Leslie Lamport in "The Part Time Parliament" and "Paxos Made Simple".

> *Early in this millennium, the Aegean island of Paxos was a thriving mercantile center.*
>
> - Leslie Lamport, "The Part Time Parliament"

The main promise A careful reader should be able to derive Paxos, implement the library contract, run a three node example, and review a production design after working through this book.

1 Preface

The usual introduction to Paxos starts too late. It starts with prepare and accept messages. Those messages then look like rules from a game whose purpose was forgotten.

We shall start earlier. We shall ask a small question.

Three machines must write one word on three pieces of paper. A machine may stop. A messenger may vanish. No machine may erase ink. How can the machines make sure that two different words are never declared final?

The answer will grow in small steps. Each step will remove one bad solution. When prepare and accept finally appear, they will have no mystery left. They are the shortest names for facts that we already need.

The style of this book is mathematical, but it is not terse. We shall compute small examples. We shall stop for exercises. We shall make mistakes on purpose. We shall keep a ledger of the facts that survive every mistake.

The implementation follows the same order. First come values and ballots. Next come promises and votes. Then come slots, leader recovery, stable storage, and a replicated state machine. At each point we ask two questions.

1. What can go wrong?
2. Which fact prevents it?

There are three complete examples.

1. The small example is a counter on three nodes. Every message fits on one page.
2. The middle example is a key value service. It includes client retry, reads, recovery, and snapshots.
3. The large example is a regional control plane. It includes five voters, many clients, large values, batching, failure domains, metrics, and capacity work.

The final parts discuss tests and measurement. The benchmark compares the Zig library with OmniPaxos 0.2.2 in Rust and a pinned LibPaxos3 core in C. The comparison uses the same number of nodes and values. It records protocol differences instead of pretending that the libraries are identical.

> **A word about certainty** Testing can reveal a broken invariant. It cannot create an invariant. We first state the reason that the code is safe. We then use tests to search for errors in our statement and in our code.

1.1 How to read the book

Parts I through III form a course in Paxos. Read them in order. Part IV is the library manual. Part V contains examples. Part VI covers engineering. Part VII is a reference for use beside an editor.

Short notes marked "Exercise" are part of the argument. A reader who answers them will remember the proof far longer than a reader who merely agrees with it. Hints appear when a calculation has a trick.

Code fragments use the public API unless they are explicitly labeled as protocol internals. Complete source files remain in the repository. Small fragments in the book may omit routine error handling only when the omitted line has already been shown.

1.2 Notation

Mark	Meaning
`A1`, `A2`, `A3`	Three acceptors.
`C`	A candidate or proposer.
`L`	A learner.

Mark	Meaning
`b = (r, n)`	A ballot with round `r` and node `n`.
`Q`	A quorum.
`s`	A one based log slot.
`v`	An application value.
`null`	No prior accepted value.

We use the word "node" for a process that may contain all Paxos roles. We use "acceptor" when only its voting duty matters. We use "leader" for a proposer that completed phase one. We use "chosen" for a fact established by a quorum. We use "committed" for a chosen value that a learner has recorded.

1.3 Commands used in the book

```
zig build fmt
zig build test
zig build run
zig build benchmark
zig build book
```

The last command creates `docs/part-time-parliament.pdf`. The benchmark command builds the pinned Rust package and fetches the pinned C source. Its first run needs network access.

Contents

PART I

One decision

We begin with one empty line in a ledger. We end with the safety rule that determines every legal Paxos message.

2 The empty ledger

Suppose that three librarians keep copies of one ledger. The next line is empty. A visitor asks them to write `olive = 7`. Another visitor asks them to write `olive = 9`. A librarian may leave the room. A messenger may lose a note. Notes that do arrive are exact.

The required result is modest. We do not require a value to be chosen at every moment. We require that if a value is chosen, no other value can ever be chosen for that line.

This distinction is our first useful tool.

Safety Nothing bad happens. For Paxos, two different values are never chosen for the same decision.

Progress Something good eventually happens. For Paxos, a proposed value is eventually chosen when a quorum can communicate and one candidate remains active long enough.

A stopped system can be safe. It is not useful, but it has not contradicted itself. This fact lets us design safety without guessing how long a message may take.

2.1 Three tempting answers

The first answer is "ask everybody." It works until one librarian leaves. One missing reply stops all work.

The second answer is "ask any two." This is better. Any two of three form a majority. Yet we have not said what a librarian remembers. If the two librarians forget their earlier answer after a restart, a later pair can choose another value.

The third answer is "let one leader decide." This moves the question. How does a new leader learn what the old leader may already have decided?

All three answers contain a piece of Paxos. We need a quorum, durable memory, and a leader recovery rule. None is sufficient alone.

Exercise 1.1. In a cluster of five nodes, how many nodes form a majority? How many may be unavailable while progress remains possible?

Hint: Compute `floor(N / 2) + 1`.

2.2 The failure model

We assume crash faults.

1. A node follows the algorithm while it runs.
2. A node may stop at any instruction.
3. A node may restart from data that reached stable storage.
4. A message may be lost, delayed, duplicated, or reordered.
5. A delivered message is not corrupted.

We do not assume that clocks agree. We do not assume a maximum message delay. We do not defend against a node that invents votes or sends two lies. That is a Byzantine problem and needs a different protocol.

The disk is part of the proof A promise that was sent and then forgotten is not a promise. If an acceptor replies before its state is durable, a restart can break quorum intersection.

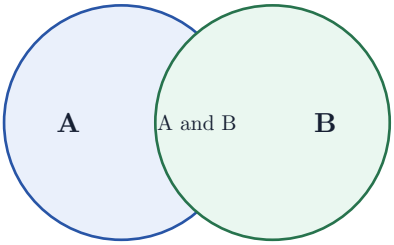


Figure 1: Two majority quorums overlap. The shared node carries knowledge from one ballot to the next.

3 Quorums

A quorum is a set large enough that every two quorums share a member. For equal voters, a strict majority has this property.

For three nodes, every quorum has two nodes. The possible quorums are `{A1, A2}`, `{A1, A3}`, and `{A2, A3}`. Pick any two of those sets. Their intersection is not empty.

For five nodes, every quorum has three nodes. Two sets of three drawn from five must share at least one node. The calculation is simple. If they did not meet, they would contain six distinct nodes, but only five exist.

3.1 Why an odd count is common

With four nodes, a majority is three. The cluster still tolerates only one unavailable node. With three nodes, a majority is two and the cluster also tolerates one. The fourth voter adds cost without adding failure tolerance.

This does not mean that every cluster must have an odd count. Failure domains, weighted quorums, and geographic placement may justify another shape. The current library uses uniform majority quorums. Its simple rule is visible in `Membership.quorum`.

```
pub fn quorum(self: *const Membership) usize {
    return @as(usize, self.count) / 2 + 1;
}
```

The division is integer division. For `count = 5`, it yields `2 + 1 = 3`.

3.2 Intersection is not memory

Let quorum `{A1, A2}` accept `x`. Later quorum `{A2, A3}` meets it at `A2`. If `A2` remembers its vote, the later quorum has a witness. If `A2` forgets, the sets still intersect on paper but no knowledge crosses the intersection.

Thus quorum intersection and stable storage form one idea. The set supplies a witness. The disk lets the witness speak after a restart.

Exercise 2.1. Construct two sets of two nodes in a four node cluster that do not intersect. Why can a quorum of two not be used there?

4 Ballots

A single leader would make the choice easy. Failures create a sequence of possible leaders. Paxos gives each attempt a ballot. Ballots are unique and have a total order.
The library uses a pair.

```
pub const Ballot = struct {
    round: u64,
    node: NodeId,
};
```

Pairs are ordered from left to right. First compare the round. If rounds are equal, compare the node.

Left	Right	Result
`(4, 2)`	`(5, 1)`	The right ballot is greater.
`(7, 1)`	`(7, 3)`	The right ballot is greater.
`(9, 4)`	`(9, 4)`	The ballots are equal.

The node component makes simultaneous ballots unique. Node 2 may use `(11, 2)`. Node 3 may use `(11, 3)`. They cannot own the same ballot.
A node that sees a higher round remembers it. Its next campaign uses a still higher round. The `u64` space is finite, so the library reports `error.BallotExhausted` rather than wrapping.

4.1 Votes and success

A ballot has four conceptual parts.
1. A ballot number.
2. A proposed value.
3. A quorum.
4. The members of that quorum that accepted the value.

A ballot succeeds when every member of its selected quorum accepts. An implementation often sends to every member and waits for any quorum. The proof is the same because the acknowledgements define the successful quorum.
The word "chosen" means that such a quorum exists. The proposer need not know that it exists. Imagine that the final acknowledgement reaches the proposer, the value becomes chosen, and the proposer crashes before announcing it. The fact remains. A future leader must preserve it.

5 Three invariants

Lamport gives three conditions. We shall state them in code language.

Rule	Statement	Mechanism
`B1`	Every ballot number is unique.	The pair `(round, node)`.
`B2`	Every two ballot quorums intersect.	Majority membership.
`B3`	A later ballot preserves the highest earlier vote seen in its phase one quorum.	Prepare, promise, and value selection.

The first two rules are easy to enforce. The third is the heart of Paxos.
Suppose a candidate starts ballot 12. It asks a quorum what each member last accepted. The replies are:

Acceptor	Accepted ballot	Value
`A1`	`8`	`red`
`A2`	`10`	`blue`
`A3`	`null`	`null`

The candidate must propose `blue` because ballot 10 is the highest reported ballot. It may not choose `green`, even if `green` was the client's request.

> **The recovery choice** If no reply contains an accepted value, use the new value. Otherwise use the value attached to the greatest accepted ballot in the quorum replies.

5.1 Why the greatest vote matters

Assume that ballot 6 chose `red`. A later successful ballot must also use `red`. Its quorum intersects the quorum for ballot 6. At least one reply has knowledge that begins with `red`. There may be intermediate ballots. The greatest prior vote reported by the new quorum carries the value that those intermediate ballots were themselves required to preserve.
This is an induction. The base case is ballot 6. The induction step says that a later ballot takes the greatest earlier vote from an intersecting quorum. It therefore cannot be the first later ballot to change the value.
We can phrase the proof as a minimal counterexample. Suppose some later ballot is the first ballot after 6 to use another value. Its quorum meets ballot 6's quorum. Every reported vote between 6 and this first bad ballot still contains `red`, by the choice of "first." The greatest report therefore contains `red`. The ballot was required to choose `red`. This contradicts the assumption that it was bad.

> **Exercise 4.1.** A phase one quorum reports `(3, apple)`, `(9, apple)`, and `(7, pear)`. Which value must the new ballot use? Does the answer change if ballot 3 is known to have succeeded?

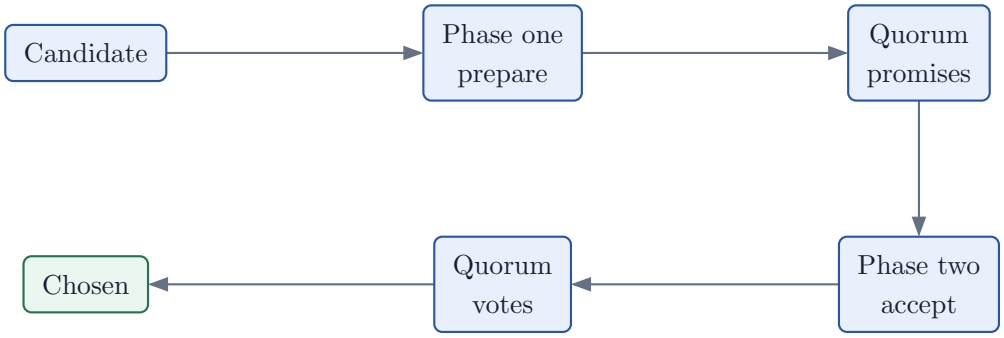


Figure 2: Phase one obtains promises and prior votes. Phase two obtains acceptances. A quorum of votes makes the value chosen.

6 From rules to messages

We now have enough facts to predict the protocol.
The candidate needs a fresh ballot. It needs a quorum to promise not to vote in older ballots. It needs each promise to include the last accepted vote. Then it can choose the required value and ask the quorum to accept it.
Those sentences give the message names.
The next part follows every message, including its disk write and rejection case. Nothing will be added merely because a textbook says so. Each field will pay rent by protecting one invariant.

PART II

The complete ballot

We execute one ballot. We stop at every write, every reply, and every failure. At the end we can recover after a crash without guessing.

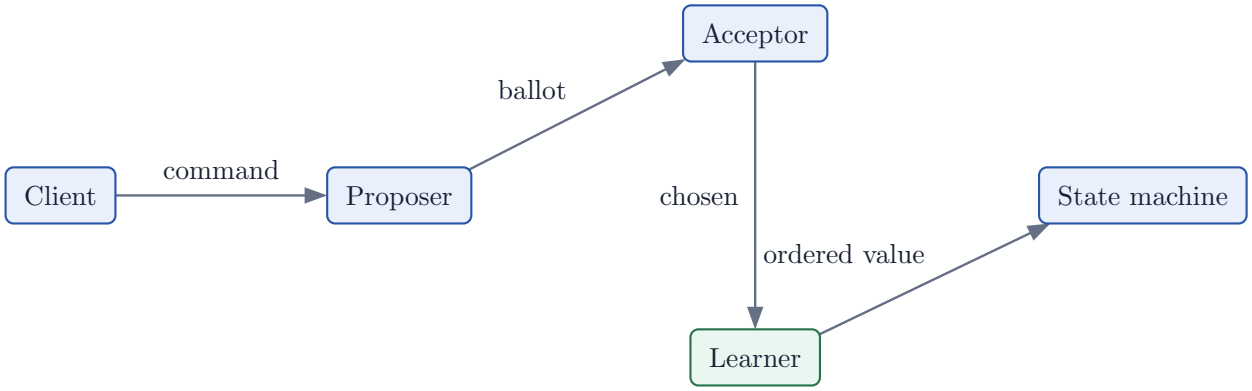


Figure 3: A client supplies intent. Paxos orders that intent. The state machine gives the intent application meaning.

7 The four roles

Paxos names proposers, acceptors, and learners. Applications add clients. One process may perform all four roles. The names describe duties, not machines.

The proposer owns one ballot number. The acceptor owns durable promises and votes. The learner owns knowledge of chosen values. A client owns the request identity that lets it retry safely.

Role	**Question**	**Durable fact**
Proposer	Which value may this ballot propose?	No proposer state is required for safety after a crash.
Acceptor	May I vote, and what did I vote for?	Highest promise and last accepted vote.
Learner	Which value was chosen?	Known commits and applied index.
Client	Did my logical request execute?	Stable client and request IDs.

8 Phase one

8.1 Prepare

The candidate chooses a ballot greater than every ballot it has attempted, promised, or observed in a rejection. It sends:

```
prepare { ballot }
```

The message contains no value. This is important. The candidate does not yet know whether it may use the client's value.

8.2 Promise

An acceptor compares the prepare ballot with its durable promise.

Comparison	Action	Reason
New ballot is lower.	Send `nack`.	An earlier promise forbids the vote.
New ballot is equal.	Repeat the reply.	The message is a duplicate.
New ballot is greater.	Persist the promise, then reply.	A reply must survive restart.

For one slot, the reply contains either no accepted vote or one pair:

```
promise {
    ballot,
    accepted_ballot,
    accepted_value,
}
```

The word "promise" has exact force. After promising ballot 11, the acceptor will not accept ballot 10. It may accept ballot 11 or 12.

> **Never reply first** If the promise reply reaches the candidate before the disk write is durable, a crash may erase the promise. Another proposer can then collect a vote that should have been forbidden.

8.3 Complete quorum replies

The candidate waits for a quorum. It does not wait for every member. Waiting for all would let one failed member stop progress.

For a single slot, each reply is one message. For the bounded Multi Paxos log, one reply may contain many accepted entries. The library sends one entry message per accepted slot and a final count. This count lets the receiver detect a final marker that arrived before an entry.

We shall return to that detail in Part III.

8.4 Choose a value

The candidate examines the complete quorum.

```
if no accepted vote was reported:
    choose the client value
else:
    choose the value from the greatest accepted ballot
```

The candidate is now a leader for this ballot. It may begin phase two.

9 Phase two

9.1 Accept

The leader sends:

```
accept {
    ballot,
    slot,
    value,
}
```

An acceptor again checks its promise. This second check is necessary. A higher prepare may have arrived after the phase one reply.

If the ballot is current or greater, the acceptor persists the ballot and value. Only then does it send:

```
accepted { ballot, slot }
```

If the ballot is lower, it sends a rejection with the highest promise it knows.

9.2 Local acceptance

The leader is also an acceptor. It need not send a network message to itself. The Zig library writes the local acceptance directly into the effect batch. It then sends accept messages only to remote peers. This saves one outbound message and still makes the write before send order explicit.

```
self.durable.promised = self.ballot;
self.durable.accepted[index] = .{
    .ballot = self.ballot,
    .value = value,
};
effects.addWrite(.{ .accept = .{
    .ballot = self.ballot,
    .slot = slot,
    .value = value,
} });
```

The local acknowledgement is then marked in memory. A three node leader needs one remote acknowledgement to reach a quorum of two. It still sends to both peers so the other replica can catch up.

9.3 Chosen and learned

When a quorum has accepted the same ballot and value, the value is chosen. The leader records a commit and sends the value to peers.

A commit message is evidence supplied by a correct Paxos participant. It does not carry a magic proof. In the crash fault model, nodes follow the algorithm, so a leader announces commit only after a quorum. A Byzantine design would need signed quorum evidence or another certificate.

10 A complete trace

Let nodes 1, 2, and 3 begin empty. Node 1 proposes `tea` with ballot `(1, 1)`.

Step	Actor	Event and reason
1	N1	Creates ballot `(1, 1)` and sends prepare to all three nodes.
2	N1	Persists promise `(1, 1)` before its local promise reply.
3	N2	Persists promise `(1, 1)` and reports no accepted value.
4	N1	Has a quorum of complete promises. No old value exists.
5	N1	Persists local acceptance of `tea` in slot 1.
6	N1	Sends accept for `tea` to N2 and N3.
7	N2	Persists the acceptance and replies accepted.
8	N1	Local N1 plus remote N2 form a quorum. `tea` is chosen.
9	N1	Persists commit and sends commit to N2 and N3.
10	All	Release slot 1 to their state machines when learned.

N3 may receive its accept after the value is already chosen. Its vote is useful for redundancy but is not needed for the fact of choice.

10.1 Message cost

Election is excluded from the steady path. For one new value on three nodes:

Step	Messages	Comment
Accept to remote peers	2	The leader accepts locally.
Accepted replies	2	One reply completes a quorum.
Commit to remote peers	2	The leader learns locally.
Total	6	No serialization or transport framing included.

11 Duplicate and reordered messages

The network is allowed to repeat every message. We therefore ask whether each handler is idempotent.

11.1 Duplicate prepare

If the ballot equals the durable promise, the acceptor repeats its report. It does not write a second promise. A reply that was lost can therefore be retried.

11.2 Duplicate accept

If the same ballot, slot, and value were already accepted, the acceptor repeats the acknowledgement without another write. If the same ballot and slot carry a different value, the library returns `error.ConflictingValue`. One unique ballot must not have two values.

11.3 Duplicate acknowledgement

Acknowledgements are stored by member index in a fixed Boolean array. Repeating one does not create a second voter.

11.4 Duplicate commit

The same value is harmless. A different value for an already committed slot is `error.ConflictingCommit`. The error marks a violated safety boundary.

11.5 An old accept arrives late

The acceptor compares it with the highest promise. A lower ballot receives a rejection. The old message cannot turn time backward.

Exercise 8.1. List every durable write in the complete trace. For each write, name the first outbound message that would be unsafe if it left before that write completed.

12 Rejection and another campaign

A rejection carries two ballots.

```
nack {
    rejected,
    promised,
}
```

The candidate acts only if `rejected` is its current ballot and `promised` is greater. It becomes a follower and remembers the observed round. Its next campaign chooses a round above that value.

The rejection itself is not a promise made by the receiving node. The receiver must not copy another node's promise into its own durable promise. The library keeps `highest_observed_round` as volatile campaign guidance.

13 Progress and the distinguished proposer

Safety does not require one leader. Two candidates may prepare higher and higher ballots forever. Each prevents the other's accept phase from finishing. No two values are chosen, but no value is chosen either. Progress needs an eventual distinguished proposer. This is usually supplied by a failure detector and randomized timeouts. The detector may be wrong for a while. That affects speed, not safety.

The library contains no clock. The host decides when to call `campaign`. This keeps time out of the safety core and lets a simulator control elections exactly.

> **The liveness assumption** A quorum can exchange messages, and after some time one candidate starts a ballot that no higher candidate interrupts.

14 Crash points

We now inspect the dangerous moments.

Crash point	Durable state	Recovery
Before promise sync	Old promise.	No promise reply was allowed to leave.
After promise sync, before reply	New promise.	A repeated prepare gets a valid reply.
Before accept sync	No new vote.	No accepted reply was allowed to leave.
After accept sync, before reply	New vote.	Phase one of a later leader discovers it.
After quorum, before commit	Votes exist on a quorum.	A later leader must recover the chosen value.
After commit sync, before send	Local commit.	Catch up sends it later.

This table is the operational form of the proof. A system that cannot explain a crash at each row is not ready to run Paxos.

15 One decision as a library instance

Classic Paxos is the bounded protocol with one slot.

```
const Synod = paxos.Protocol(Command, .{
    .max_members = 3,
    .max_slots = 1,
});
```

Campaign once. Propose once. After slot 1, `propose` returns `error.SlotLimitReached`. The next part shows how independent decisions form a log and how one successful phase one can serve many of them.

PART III

A sequence of decisions

A service needs more than one chosen value. We arrange decisions in slots, recover an old leader's work, fill holes, and apply one ordered prefix.

16 One Paxos instance per slot

Let slot 1 choose `open`. Let slot 2 choose `write A`. Let slot 3 choose `close`. Each slot is a separate consensus problem. Safety for slot 2 says nothing about slot 3. The log obtains its order from slot numbers.

The direct construction runs phase one and phase two in every slot. It is safe and slow. A stable leader can prepare one ballot across all bounded slots. Once it has a quorum of complete replies, it may run phase two for each free slot. This is Multi Paxos.

The proof has not changed. Each slot still follows B1, B2, and B3. We have only combined the phase one questions into fewer messages.

16.1 Slot numbers

The library uses one based `u32` slots. Slot zero means "no slot." The conversion to an array index occurs in one helper.

```
fn slotIndex(slot: Slot) !usize {
    if (slot == 0) return error.InvalidSlot;
    if (slot > options.max_slots) return error.SlotLimitReached;
    return @as(usize, slot - 1);
}
```

Centralizing the conversion is more than tidiness. It gives one place to check the two boundaries. An index is not a count. A slot is not an index.

17 A batched prepare reply

An acceptor may have accepted values in many slots. Its response to prepare is a series of messages:

```
promise(ballot, slot 2, accepted ballot 7, value B)
promise(ballot, slot 5, accepted ballot 9, value E)
promise_done(ballot, accepted_count 2)
```

The network may deliver `promise_done` first. If the candidate counted the member immediately, it could become leader without seeing value `E`. That value might already be chosen.

The candidate therefore tracks, for each member:

1. whether the final marker arrived,
2. the number of entries promised by the marker,
3. the number of distinct entries received,
4. a Boolean per slot to reject duplicates.

A member's response is complete only when the marker exists and both counts are equal. A quorum means a quorum of complete responses.

> **Why a count instead of FIFO** TCP preserves sender order on one connection, but the Paxos safety core need not assume TCP. A count makes the requirement explicit and lets other transports reorder messages safely.

17.1 Selecting recovered values

For each slot, compare every accepted ballot reported by the complete quorum. Keep the greatest one and its value.

Consider three promise replies.

Member	Slot	Ballot	Value
`N1`	`4`	`(6, 1)`	`alpha`
`N2`	`4`	`(8, 2)`	`beta`
`N3`	`4`	`null`	`null`

The new leader must propose `beta` in slot 4. It does not matter that `alpha` arrived first. Arrival order is not ballot order.

The test named "phase one tolerates reordering and recovers the highest accepted value" constructs this schedule. It sends completion markers before entries and gives two minority acceptors different old values. The leader waits and chooses the value from the greater ballot.

18 Holes and the no op value

Suppose recovery finds a value in slot 5 but no accepted value in slot 4. The leader must not put an important new command into slot 4 if clients may already have observed work associated with slot 5. It fills the hole with a no op.

Lamport called this the olive day decree. It changes no application state. It does make the log prefix complete.

The host supplies the no op value when it calls `campaign`.

```
try node.campaign(.{
    .client_id = 0,
    .request_id = 0,
    .operation = .noop,
}, &effects);
```

The protocol cannot invent a valid generic `Value`. Only the application knows which value means no work.

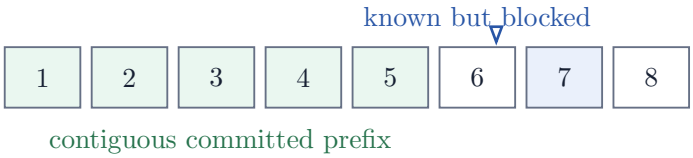


Figure 4: Slots 1 through 5 may be applied. Slot 7 is known, but slot 6 blocks it.

19 Learning in order

A network can deliver commit for slot 7 before commit for slot 6. The learner records slot 7 but does not apply it. State machines must see one common prefix.

When slot 6 arrives, the learner emits both 6 and 7 in order. The field `delivered_through` marks the prefix already returned during this process lifetime.

Across process restart, delivery is at least once. The application must persist its applied slot with its state. If it sees an old slot again, it ignores it.

19.1 Catch up

A lagging node sends:

```
learn { from_slot }
```

The peer returns every known commit at or after that slot. The method is simple and bounded. It is suitable for the library's fixed log. A large production system should transfer a snapshot when the missing prefix is too large.

The catch up request may go to any member. A leader is a good choice because it is likely to know the newest prefix. A stale peer can return only what it knows; the learner may ask another peer later.

20 Stable leader pipeline

After phase one, several slots may be in phase two at the same time. The API lets the host call `propose` again before an earlier proposal commits. It can also use `proposeBatch` to assign several consecutive slots in one call. Each slot has its own compact acknowledgement bit set.

Pipelining hides network latency, but it creates a bound question. The current library bounds slots and effect buffers at compile time. The host should also bound client requests in flight. An unbounded input queue would move the memory problem outside the protocol without solving it.

For a three node cluster, one stable value creates six remote protocol messages. With `W` values in flight, a rough upper bound for protocol envelopes in the transport is `6W`, before retries and catch up. The right `W` follows from disk latency, network bandwidth, and the largest acceptable response time.

21 Leader replacement trace

We now follow the case that makes Paxos worth learning.

Step	Actor	Event and reason
1	N1	Leads ballot `(4, 1)` and sends accept for `red` in slot 8.
2	N2	Persists the acceptance and replies.
3	N1	The local vote and N2 form a quorum. `red` is chosen.
4	N1	Crashes before sending commit.
5	N3	Starts ballot `(5, 3)` and sends prepare.
6	N2	Promises `(5, 3)` and reports `(4, 1), red` for slot 8.
7	N3	Gets a complete quorum. `red` is the greatest report for slot 8.
8	N3	Reproposes `red` in ballot `(5, 3)`.
9	N3	Gets a quorum and announces commit.

The client may have timed out at step 4. It cannot conclude that `red` failed. It must retry with the same request identity. The state machine will recognize the duplicate if a second slot later contains that request.

21.1 A value accepted by only one node

Now suppose N1 crashes before its local vote gains any remote vote. The value is not chosen. A later leader may still recover it if N1 belongs to the new phase one quorum and its ballot is the greatest report. Preserving the value is conservative. It is not evidence that the old client succeeded. It is the uniform rule that also protects values which did succeed without an announcement.

22 Reads

Consensus orders writes. Reads need a stated consistency level.

Read	Method	Property
Local	Read the local applied state.	Fast and possibly stale.
Log barrier	Propose a read marker and wait to apply it.	Linearizable and consumes a slot.
Read index	Confirm current leadership with a quorum, then wait for the confirmed applied index.	Linearizable with extra protocol support.
Lease	Use a time bounded leader lease.	Fast, but clocks enter the safety argument.

Version 0.1 supplies local decided reads and the log barrier through ordinary proposals. It does not implement read index or leases. The application must not call a local read linearizable merely because it came from the leader.

23 Membership

Membership changes are consensus decisions about future quorums. A careless switch from old members to new members can create two nonintersecting quorums.

The small `Protocol.Node` keeps membership fixed. The `ReplicatedLog.Node` places a stop sign in the ordered log. The stop sign names the next membership and seals the current configuration. A decided stop sign constructs the next epoch. The transport must retain enough configuration identity to reject delayed messages.

Other choices include joint consensus, delayed activation by slot, and matchmaker protocols. They differ in proof and operation. This library selects one explicit stop sign design rather than hiding several proofs behind a Boolean option.

24 Bounded logs and epochs

`max_slots` is a hard limit. A slot is never reused within an epoch. After the last slot, `propose` returns `error.SlotLimitReached`.

Before that point, the application should:

1. persist an application snapshot and its applied slot,
2. verify the snapshot on enough nodes,
3. call `checkpoint` with an immutable snapshot reference,
4. decide the stop sign and stop proposals in the old epoch,
5. create the next epoch with `initFromStop`,
6. reject delayed envelopes from the old epoch at the transport boundary.

The core message type does not contain a configuration identifier. A production transport must frame one outside the message. Node IDs should also remain stable for the logical members they name.

PART IV

The Zig library

The proof becomes a bounded state machine. The host owns disk, network, time, and application state. The boundary between them is the main API.

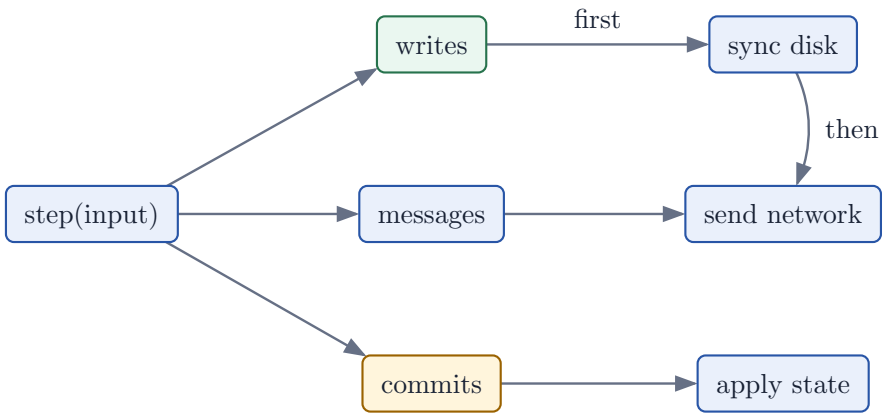


Figure 5: One input creates three kinds of effects. Durable writes are consumed first.

25 Design before syntax

The library performs no input or output. It starts no thread. It reads no clock. It allocates no memory after initialization. A node consumes one input and fills one caller supplied effect buffer.

This design is useful for three reasons.

1. The same core works inside TCP, QUIC, an actor runtime, or a simulator.
2. A test can choose every delivery order and crash point.
3. The write before send rule is visible at the call site.

The cost is also visible. The host must supply real storage, a transport, tick delivery, serialization, snapshots, and a deterministic state machine. The library is a consensus component. It is not a server process.

26 Package use

For a tagged Codeberg release, ask Zig to compute the archive hash.

```
zig fetch --save \
  https://codeberg.org/OWNER/paxos/archive/v0.1.0.tar.gz
```

The consumer's `build.zig` exposes the module.

```
const dependency = b.dependency("paxos", .{
    .target = target,
    .optimize = optimize,
});

const application = b.addExecutable(.{
    .name = "ledger",
    .root_module = b.createModule(.{
        .root_source_file = b.path("src/main.zig"),
        .target = target,
        .optimize = optimize,
        .imports = &.{.{
            .name = "paxos",
            .module = dependency.module("paxos"),
        }},
    }),
});
```

For local work, use `.paxos = .{ .path = "../paxos" }` in `build.zig.zon`. The fixture under `integration/consumer` compiles this exact arrangement.

27 Choose the value and bounds

```
const paxos = @import("paxos");

const Command = struct {
    client_id: u128,
    request_id: u64,
    operation: Operation,
    key: [32]u8,
    value_hash: [32]u8,
};

const Consensus = paxos.Protocol(Command, .{
    .max_members = 5,
    .max_slots = 16_384,
});
```

The command is copied into node state and messages. A borrowed slice would copy only a pointer and a length. That pointer might die before a message is encoded. To guarantee safety, the library enforces at compile time (via metaprogramming reflection) that the `Value` type contains no pointers, slices, or references. Any attempt to instantiate a protocol with a pointer-containing type will trigger a compilation error. Use fixed-size arrays, inline values, or content hashes referring to separate durable blob storage.

Bounds are compile-time values. They determine the size of nodes and effect buffers. Large bounds can create large stack objects. To avoid compilation bottlenecks on large bitsets, the library optimizes its bitset representation: bitsets of size at most 64 (such as `MemberSet` on typical clusters) are backed by a single register-sized unsigned integer, whereas larger bitsets (such as `SlotSet` with thousands of slots) are backed by an array of 64-bit words. This hybrid design ensures maximum CPU shifting speed and prevents compiler exhaustion. Long-lived nodes should normally live in stable application storage rather than in a deeply nested call.

27.1 Capacity calculation

Let `M` be members and `S` be slots. The important arrays are proportional to:

State	Order	Purpose
Accepted and committed values	`O(S)`	Durable protocol history.
Promise receipt bits	`O(MS)` bits	Duplicate safe phase one recovery.
Acknowledgement bits	`O(MS)` bits	One vote per member and slot.
Largest recovery output	`O(MS)`	Accept broadcast for recovered slots.

Compile a representative configuration and inspect `@sizeOf(Consensus.Node)`. Do not select one million slots merely because the type accepts the number.

28 In place construction

The node is large. TigerStyle recommends construction in place so that an accidental stack copy cannot hide in a return value.

```
var membership: Consensus.Membership = undefined;
try membership.init(&.{ 1, 2, 3 });

var node: Consensus.Node = undefined;
try node.init(1, &membership);
```

Membership rejects an empty set, zero IDs, duplicates, and a set above the compile time bound. A node ID must belong to its membership.

The membership is copied intentionally into the node. The source is passed by pointer because it may exceed 16 bytes and because the call site should not make an accidental temporary copy.

29 The public types

29.1 Ballot

`Ballot` provides `order`, `lessThan`, and `eql`. Ballot zero is the initial sentinel. Real node IDs are nonzero.

29.2 Message and Envelope

`Message` is a tagged union. `Envelope` adds source and target node IDs. The transport encodes the tag and payload. It must reject frames that exceed a bound before allocating or decoding their value.

29.3 DurableState

`DurableState` contains:

1. the highest promised ballot,
2. an optional accepted ballot and value for every slot,
3. an optional committed value for every slot.

`DurableState.apply(write)` is a reference journal replay function. It rejects a promise that moves backward and a commit that conflicts with an earlier commit. It asserts that no accepted ballot exceeds the highest promise.

29.4 Effects

`Effects` contains fixed arrays and active counts. Initialize it in place once.

```
var effects: Consensus.Effects = undefined;
effects.init();
```

A public operation resets the counts. The caller must consume the batch before calling the node again. Reuse the same buffer in the event loop. Constructing a large buffer inside a hot function can clear far more memory than the active effect entries require.

The arrays remain uninitialized beyond their active counts. Serialization must use the slices, never the entire backing arrays. This avoids reading padding or unused memory.

30 The effect contract

Every call follows one order.

```
try node.step(envelope, &effects);

for (effects.writesSlice()) |write| {
    try journal.append(write);
}
try journal.sync();

for (effects.messagesSlice()) |message| {
    try transport.send(message);
}

for (effects.committedSlice()) |entry| {
    try machine.apply(entry.slot, entry.value);
}
```

Writes may be grouped in one atomic batch. Messages from that operation wait for the batch. Commits also wait if the application state is meant to reflect only durable protocol knowledge.

If append or sync fails, stop the node. Restore it from the last good journal. Do not continue with the advanced in memory state.

> **A common integration bug** Putting writes and sends on independent queues does not preserve the contract. The send queue may win. Use a barrier or one owner that releases messages only after the durable completion arrives.

31 Campaign

```
try node.campaign(noop, &effects);
```

The call chooses a round greater than the attempted round, local promise, and highest observed rejection. It clears volatile election state and broadcasts a prepare.

The no op is retained for holes discovered by recovery. Campaign does not make the node a leader. The node becomes a leader only after `step` has processed a complete quorum of promise replies.

Multiple calls are safe. They create higher ballots and may harm progress. The usual host calls `tick` and lets the configured logical timeout start a campaign.

32 Propose

```
const slot = try node.propose(command, &effects);
```

The node must be a prepared leader. The call assigns `next_slot`, records the local acceptance, places its durable write in effects, and sends accept messages to remote peers. The returned slot is an address, not a success result.

When the last bounded slot is used, `next_slot` becomes zero. The next proposal returns `error.SlotLimitReached`. Saturating arithmetic is not used because it could reuse the final slot.

33 Step

`step` accepts one envelope. It rejects a wrong recipient and a source outside membership. The message tag selects a small handler.

Message	Main state	Possible output
`prepare`	Promise and accepted log.	`promise`, `promise_done`, or `nack`.
`promise`	Recovery candidate per slot.	Leader recovery when complete.
`promise_done`	Expected entry count.	Leader recovery when complete.
`accept`	Promise and accepted value.	`accepted` or `nack`.
`accepted`	Member bitmap per slot.	`commit` after quorum.
`commit`	Committed value and prefix.	Contiguous application entries.
`learn`	Known commits.	`commit` replies.
`nack`	Observed round and role.	No immediate output.
`heartbeat`	Leader contact and ballot.	`nack` if stale.

The parent `step` owns the branch. Leaf handlers perform one protocol duty. This keeps control flow flat and each function below the TigerStyle 70 line limit.

34 Request catch up

```
try node.requestCatchUp(peer, first_missing_slot, &effects);
```

The peer must be in membership and the slot must be nonzero. The method emits a single learn request. The caller can retry another peer after its own timeout.

35 Restart

Journal replay builds a `DurableState`. Restore writes into an existing node.

```
var durable: Consensus.DurableState = .{};
for (records) |record| try durable.apply(record);

var node: Consensus.Node = undefined;
try node.restore(my_id, &membership, &durable);
```

Volatile leadership, acknowledgements, proposals, and partial promise replies are discarded. The node restarts as a follower. This is safe because a later campaign recovers accepted state from a quorum.

36 Serialization

The generic library does not prescribe bytes. A production frame should include:

Field	Purpose
Magic and version	Reject unrelated or incompatible frames.
Cluster and epoch ID	Reject delayed traffic from another deployment.
Source and target	Authenticate routing identity.
Message tag	Select the union payload.
Payload length	Bound decode work before allocation.
Checksum or MAC	Detect corruption or authenticate the sender.

Integer byte order must be fixed. Unknown versions and tags must be rejected. Do not cast a network byte slice directly to a Zig struct. Padding and host byte order are not a wire format.

37 Storage format

Journal records should be framed separately from network records. One practical layout is:

```
record_length | record_version | record_tag | payload | checksum
```

On recovery, scan from the beginning or from a trusted checkpoint. Verify each length and checksum. Stop at a torn final record. Reject corruption in the middle. Apply records in order.

The applied state machine index belongs in the application snapshot. Persist the snapshot and index atomically. Protocol commit delivery after restart is at least once, so the index prevents a second application.

38 Errors and assertions

Operating errors are returned. Examples are invalid membership, wrong recipient, not leader, and slot exhaustion. Programmer and invariant errors are assertions.

The core asserts that:

1. the local node remains a member,
2. counts remain inside their arrays,
3. accepted ballots do not exceed the promise,
4. slots and delivered prefixes remain bounded,
5. a local accept never goes below the promise.

Assertions turn silent state corruption into a stopped node. In consensus, a stopped node is usually safer than a node that continues with impossible state.

39 TigerStyle audit

The implementation applies the relevant rules as design constraints.

Rule	Application
Bound everything.	Members, slots, messages, writes, commits, and loops have compile time bounds.
No allocation after init.	The consensus core contains no allocator.
Simple control flow.	No recursion. Message dispatch is one explicit switch.
Assert invariants.	Public boundaries and buffer mutations use paired assertions.
Keep functions short.	Core functions remain below 70 lines.
Keep lines short.	The format check enforces at most 100 columns.
Initialize large values in place.	`Membership`, `Node`, and restore use destination pointers.
Explain why.	Comments describe safety reasons rather than restating syntax.
Zero runtime dependencies.	The published Zig module imports only Zig std.

`usize` appears where Zig slices and arrays require an index or length. Protocol identities and counters that cross the API use explicit `u32`, `u64`, or `u16` types. This is a deliberate boundary between machine indexing and protocol data.

40 The complete log interface

The small `Protocol` type exposes the proof. The `ReplicatedLog` type exposes the shape most applications need. It orders commands, seals configurations, carries snapshot references, and keeps every storage decision bounded.

This chapter is a tour of the complete interface. We shall begin with time. We shall end with a new parliament.

40.1 One owner and one reusable buffer

A node should have one owner. The owner may be an actor, an event loop task, or a thread. It supplies one effect buffer and reuses it for every call.

```
var effects: Log.Effects = undefined;
effects.init();

try node.tick(noop, &effects);
try consume(&effects);

try node.step(envelope, &effects);
try consume(&effects);
```

The call to `init` changes only three counts. It does not clear the large fixed arrays behind them. Every public node operation later calls `reset`, which also changes only the counts.

> **Do not construct the buffer in a hot loop** A bounded effect buffer can be large. Writing `var effects = Effects{}` inside a per message function may cause the compiler to clear the complete object. Construct it once, call `init`, and pass its address. A profiler found this exact mistake in the first benchmark.

The owner consumes the effects before the next call. It never keeps a slice from an old effect batch. The next operation is allowed to overwrite those entries.

40.2 Logical time

The library reads no wall clock. The host calls `tick` at a stable interval. The interval might be ten milliseconds. It might be one simulator step. Paxos cares about the order of calls, not the name of the clock.

```
try node.tick(noop_command, &effects);
```

A follower counts ticks since useful leader contact. At `election_timeout_ticks` it starts a campaign. A leader uses the same call to send heartbeats and to scan its bounded log for work that should be resent. The three intervals have different purposes.

Option	Default	Meaning
`election_timeout_ticks`	10	A follower campaigns after this much silence.
`heartbeat_interval_ticks`	3	A leader reminds followers that its ballot is active.
`resend_interval_ticks`	10	A leader repairs missing accept and commit traffic.

The host should not call every node at exactly the same instant forever. Give preferred nodes a greater priority. Stagger process starts. A simulator should also test repeated ties. Safety does not depend on a lucky schedule. Prompt election does.

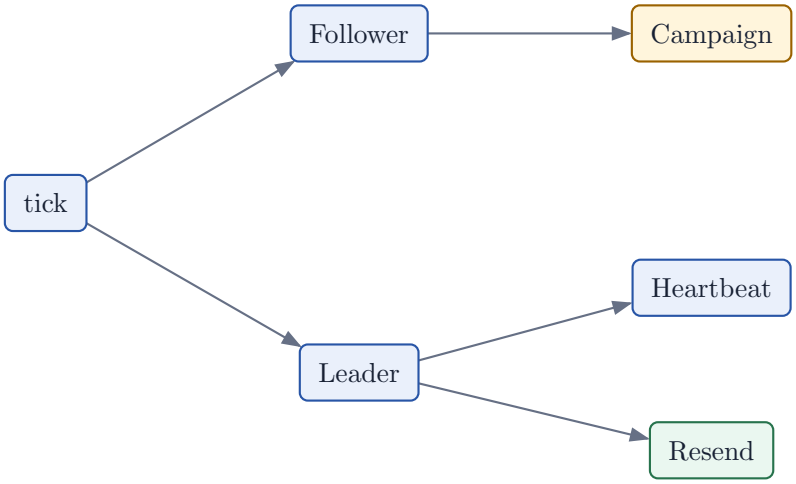


Figure 6: One logical tick drives three explicit duties. Only one branch is active for the current role.

40.3 Ballot priority

A ballot contains a round, a priority, and a node ID. Comparison uses that order.

`(round, priority, node)`

The round is most important. No priority can defeat a later round. Priority breaks a tie between candidates that begin the same round. The node ID breaks the final tie and makes every ballot unique.

```
try node.initWithPriority(my_id, &membership, 100);
```

Choose priority from a stable operational fact, such as a preferred region or a machine class. Do not change it on every tick. An unstable preference creates work and explains nothing.

40.4 Heartbeats

A heartbeat names the active ballot. It contains no command. A follower rejects an older heartbeat and observes a current one as leader contact. Accept and commit messages also count as leader contact, so a busy cluster does not need a heartbeat for every interval.

A heartbeat is not a lease. It does not prove that the sender still owns a quorum at the moment an application read begins. A linearizable read must use a protocol that confirms leadership with a quorum, or it must be represented by a log entry. The local read methods in this library report decided local state.

40.5 Retransmission

At the resend interval, a leader walks the bounded slot array. For each peer it sends known commits and active accepts. This is deliberately simple. It has a finite upper bound and it repairs lost traffic without a second data structure.

The scan costs `members * slots` in the worst case. Choose `max_entries` with that fact in mind. A host with a very large retained log can call `reconnected` when a transport returns and can use checkpoints before periodic scans become expensive.

```
try node.reconnected(peer_id, &effects);
```

A leader resends that peer's known state. A follower that reconnects to its leader asks for commits after its delivered prefix. The transport owns backoff, connection state, packet size, and congestion control.

40.6 Flexible quorums

Classic Paxos usually uses a majority in both phases. This library also accepts different read and write quorum sizes.

```
const P = paxos.Protocol(Command, .{
    .max_members = 5,
    .max_slots = 8192,
    .read_quorum_size = 4,
```

```
    .write_quorum_size = 2,
});
```

The word read here means phase one. It does not mean an application query. The word write means phase two.

For `N` members, safety requires:

```
read quorum + write quorum > N
```

Every phase one quorum must intersect every phase two quorum. Two phase two quorums need not intersect for the same leader ballot because an acceptor will not accept two values for one ballot and slot. Leader recovery is the operation that must meet an earlier chosen value.

N	Read	Write	Use
3	2	2	The ordinary majority rule.
5	4	2	Cheap steady writes and expensive leader replacement.
5	2	4	Expensive steady writes and cheaper leader replacement.
5	3	3	The ordinary majority rule.

The membership constructor rejects zero sizes, sizes above membership, and a pair that does not intersect. This turns a proof obligation into a checked configuration.

40.7 Batch proposal

`proposeBatch` assigns consecutive slots and returns one combined effect batch.

```
var slots: [32]paxos.Slot = undefined;
const assigned = try node.proposeBatch(commands, &slots, &effects);
```

The caller owns `slots`. The library allocates nothing. Every command still has its own Paxos slot and its own accept envelope. The combined effect batch lets a host encode several envelopes into one network packet and place several durable records in one storage transaction.

The `ReplicatedLog` wrapper calls this operation `appendBatch`. Its `max_batch` option bounds temporary entry conversion.

Batching has three different meanings. Keep them separate.

Kind	Meaning
API batching	One call assigns several slots and fills one effect buffer.
Storage batching	One journal transaction or sync covers several writes.
Wire batching	One transport frame carries several envelopes.

The library supplies the first. Its effect boundary makes the second and third possible. The host chooses their byte format and latency limit.

40.8 Compact vote sets

Duplicate votes must not count twice. The direct representation is one Boolean per member and slot. The C implementations studied for this library suggested a smaller representation: a bit vector.

`BitSet(K)` uses exactly `K` bits. Insertion sets one bit and reports whether the set changed. Quorum counting uses a population count. Phase one receipt sets use one bit per slot. Phase two vote sets use one bit per member.

This design has four useful properties.

1. no allocator is needed,
2. duplicate detection is constant time,
3. storage is compact and contiguous,
4. the code still says `insert` and `count` instead of exposing shifts everywhere.

The abstraction is small because readability counts. An optimization that forces every protocol handler to repeat a bit mask expression is too expensive to maintain.

Figure 7: A stop sign is the final ordered fact of one configuration and the first trusted fact used to construct the next.

41 Reconfiguration by stop sign

A fixed membership is easy to reason about. A changing membership is not. The safe method used here gives each membership its own configuration and puts one special value at the end.
That value is a stop sign.

> **Stop sign** A log entry that names the next configuration and optional snapshot metadata. Once accepted, the current `ReplicatedLog` remains sealed. Once decided, the next configuration may be initialized from it.

The entry contains:
1. a strictly greater configuration ID,
2. a bounded member list,
3. a bounded metadata byte string.

```
const slot = try node.reconfigure(
    42,
    &.{ 2, 3, 4, 5, 6 },
    "snapshot:sha256:...",
    &effects,
);
```

The metadata is protocol data. Keep it small. It may name a snapshot, schema, encryption key, or deployment record. Large snapshot bytes belong in an object store or a transfer protocol.

41.1 Why the log seals on acceptance

Suppose a leader accepts a stop sign locally and then loses leadership. The stop sign may or may not already be chosen. If the old node later appends commands beyond it, recovery must distinguish two incompatible ideas of where the configuration ended.
The wrapper chooses the simple rule. An accepted stop sign seals the local log. This remains true after journal replay. The node may still process protocol messages and help finish recovery. It may not accept new application appends in that configuration.
This rule can be conservative. A stop sign that never becomes chosen can leave one process sealed until the deployment resolves the configuration transition. The gain is a clear invariant: no local command is proposed beyond a stop that the same durable state remembers.

41.2 Starting the next configuration

After the stop sign is decided and its preceding prefix is applied, construct a fresh node.

```
const stop = node.isReconfigured().?;

var next_membership: Log.Membership = undefined;
var next: Log.Node = undefined;
try next.initFromStop(
    my_id,
    &stop,
    &next_membership,
    leader_priority,
);
```

A removed node cannot initialize because its ID is not in the new membership. The old node and old journal should remain available until the new configuration has installed the required state and the operator's retention rule permits deletion.

Messages need a configuration ID in their outer frame. The generic core does not add one to `Envelope` because deployments already need a cluster ID, wire version, and authenticated sender. The decoder must route a message to the node for exactly that configuration. A delayed packet from configuration 41 must not enter configuration 42.

41.3 Checkpoint epochs

The protocol log is bounded. Before its final slot, create an application snapshot and seal the epoch with `checkpoint`.

```
_ = try node.checkpoint(snapshot_reference, &effects);
```

Checkpoint keeps the same member list and increments the configuration ID. The stop metadata names the snapshot. The next node begins with an empty bounded protocol log while the application begins with the installed snapshot state.

The safe order is:

1. apply all decided commands through slot `K`,
2. write a snapshot that includes state and `K`,
3. make the snapshot durable and retrievable,
4. propose a checkpoint that names it,
5. wait until the stop sign and its prefix are decided,
6. initialize the next epoch from that stop sign,
7. discard old data only after the retention rule is satisfied.

The snapshot reference must identify immutable bytes. A path whose contents can change is not an identity.

42 Reads and catch up

The core offers three local views.

Method	Answer
`committedAt(slot)`	The locally known decided value for one valid slot.
`decidedThrough()`	The highest contiguous slot released to the application.
`readDecided(from, out)`	A caller owned copy of a decided contiguous suffix.

These are local reads. They do not contact a quorum. A follower may be behind. For a stale tolerant endpoint, return the local prefix and its slot. For a linearizable endpoint, use a quorum confirmed leadership method or put the read barrier in the log.

A missing follower asks a peer for commits.

```
try node.requestCatchUp(peer, first_missing_slot, &effects);
```

The peer returns each known commit from that slot. The requester releases values only when the prefix is contiguous. If the gap is large, transfer a checkpoint instead of replaying the complete retained log.

43 What remains with the host

The library is complete as a bounded consensus state machine. A production system is larger. The following duties remain explicit.

Duty	**Required host rule**
Journal	Append all writes in order and sync before any related send.
Transport	Authenticate source, bound frames, retry, and apply backpressure.
Encoding	Use a versioned byte format independent of Zig struct layout.
Clock	Call `tick` at a documented interval and test delayed calls.
Snapshots	Atomically bind application state to its applied slot.
Clients	Use stable request IDs and deduplicate applied commands.
Configuration	Route every frame by cluster and configuration identity.
Observability	Record ballot, role, leader, prefix, queue depth, and errors.

OmniPaxos includes more policy inside its Rust object, including a ballot leader election protocol with a published partial connectivity progress argument, internal batch accept messages, and live log trimming. Paxos Zig keeps policy at the effect boundary, uses checkpoint epochs for compaction, and does not claim the same partial connectivity guarantee. `docs/features.md` records each difference. A feature table is more useful than a broad claim of equivalence.

44 A compact operating recipe

The complete loop can now be stated in ten short sentences.

1. Construct membership and node in place.
2. Construct one effect buffer in place and call `init` once.
3. Give one owner all calls to that node.
4. Feed authenticated envelopes to `step`.
5. Call `tick` at the chosen interval.
6. Persist and sync writes before sending messages.
7. Apply committed entries once and in slot order.
8. Use request IDs for client retries.
9. Checkpoint before the bounded log is full.
10. Start a new configuration only from a decided stop sign.

Each sentence can be tested. That is the point. A consensus library becomes usable when its obligations fit in the caller's field of view.

45 The coding standard

A consensus library is read under pressure. A reviewer may be tracing a safety failure at three in the morning. A future maintainer may know Zig but not Paxos. The code must help both readers.
This project begins with seven sentences.

> *Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex. Complex is better than complicated. Flat is better than nested. Sparse is better than dense. Readability counts.*
>
> - Project design principles

These sentences are not permission to prefer pretty code over correct code. They tell us how correct code should be presented. TigerStyle supplies the hard bounds. Paxos supplies the invariants. Zig supplies explicit values and errors.
The order of concern is:
1. safety,
2. performance,
3. developer experience.

An attractive abstraction that hides a disk ordering rule fails the first item. A fast loop without a bound fails the first item as well. A safe and bounded function with poor names fails the third item and will eventually threaten the first two.

45.1 Readability is a safety property

Consider a promise check written as one dense expression.

```
if (done[i] and received[i] == expected[i] and ballot.eql(active)) count += 1;
```

The expression is short. It is not simple. Three facts have been compressed into one line. A reviewer must separate them mentally.
The library uses the flat form.

```
if (!promise_done[member]) continue;
if (promise_received[member] != promise_expected[member]) continue;
complete += 1;
```

Each line answers one question. Did the completion marker arrive? Did every promised entry arrive? If both answers are yes, this member is complete. Message reordering is now visible in the shape of the code.

> **The test for a readable line** Ask whether a reviewer can name the invariant protected by the line without expanding a hidden abstraction or evaluating a compound expression.

46 Control flow

Protocol control flow is deliberately ordinary.

1. Public methods validate, reset effects, perform one transition, and validate.
2. Message dispatch uses one explicit switch.
3. Invalid input returns early.
4. Loops have visible bounds.
5. Recursion is not used.
6. A helper has one protocol purpose.

This is preferred:

```
if (message.ballot.lessThan(self.durable.promised)) {
    self.sendNack(from, message.ballot, effects);
    return;
}
```

The rejection is a complete branch. The normal path remains flat. A nested version would make the common transition harder to see.

46.1 Function size

Functions are limited to 70 lines. Source lines are limited to 100 columns. These are not aesthetic guesses. A short function can usually be held in one view and reviewed as one transition. A short line leaves room for an editor, line numbers, and a debugging pane.

The generic `Protocol` and `ReplicatedLog` type factories are namespaces written as Zig functions. They are exempt from the outer function limit. Every function inside them is still checked.

If a function grows, split it by protocol responsibility. Do not split it at an arbitrary line count. `sendAccept`, `recordCommit`, and `emitContiguous` are separate because acceptance, durable choice, and application delivery are different facts.

47 Bounds and memory

The consensus path allocates no memory after initialization. Members, retained entries, batches, messages, writes, and delivery buffers have compile time bounds.

Quantity	Type	Reason
Node identity	`u32`	Stable protocol identity.
Ballot round	`u64`	Explicit exhaustion boundary.
Slot	`u32`	Wire and durable log position.
Array index	`usize`	Required by Zig slices.
Member count	`u16`	Bounded by the configured maximum.

`usize` does not cross the protocol boundary. It is used for array indexes and slice lengths. A node ID is not an index. A slot is not an index. Converting one to the other happens in a small checked function. Large structures are initialized through destination pointers.

```
var membership: P.Membership = undefined;
try membership.init(&.{ 1, 2, 3 });

var node: P.Node = undefined;
try node.init(1, &membership);
```

Returning a large node by value would make an accidental copy easy to miss. The destination pointer makes ownership and storage visible.

47.1 Batches remain bounded

Batching is useful only when it does not turn latency into an unbounded queue. `max_batch` is a compile time limit. `appendBatch` rejects a larger slice before it mutates the log. The caller supplies the slot output buffer.

Preflight checks happen before the first proposal. A rejected batch therefore has no partial prefix.

48 Assertions and errors

An error reports a state that a caller, network, or bounded resource may produce. An assertion reports a broken internal invariant.

Situation	Mechanism	Example
Invalid caller input	Return an error.	`error.InvalidSlot`
Capacity reached	Return an error.	`error.SlotLimitReached`
Stale network ballot	Return a protocol message.	`nack`
Impossible buffer count	Assert.	`count <= buffer.len`
Durable storage failure	Stop the node.	Restore from durable state.

Buffer mutations use paired assertions.

```
std.debug.assert(self.messages_count < self.messages.len);
self.messages[self.messages_count] = envelope;
self.messages_count += 1;
std.debug.assert(self.messages_count <= self.messages.len);
```

The first assertion protects the write. The second records the new invariant. ReleaseFast removes full state validation from the hot path. Debug and ReleaseSafe retain it.

The library does not use `catch unreachable` for ordinary errors. It does not panic because a peer sent an old ballot. It does not continue after a journal sync fails.

49 Names carry the proof

Names come from the domain.

1. `promised` is the ballot below which an acceptor will not vote.
2. `accepted` is a durable vote for one slot and value.
3. `committed` is a learned chosen value.
4. `delivered_through` is the contiguous prefix released to the application.
5. `readQuorum` is the phase one quorum.
6. `writeQuorum` is the phase two quorum.

Names such as `state2`, `tmp`, and `data` force the reader to rediscover facts that the program already knows. Short loop names such as `index` and `member` are acceptable when their domain is immediate.

Units belong in names when two units could be confused. Use `timeout_ticks`, `metadata_count`, and `from_slot`. Do not make the reader guess whether a number is bytes, entries, ticks, or nodes.

50 Comments explain why

This comment is weak.

```
// Increment the count.
count += 1;
```

The statement already says that. A useful comment preserves a reason.

```
// A completion marker may overtake entries on a reordering network.
// Count this member only after every announced entry has arrived.
```

Comments should mention crash boundaries, quorum arguments, ownership, or a surprising performance decision. They should not translate Zig into English.

51 Formatting is executable

The repository makes the standard part of the build.

```
zig build fmt
```

This command runs `zig fmt --check`. It also runs `tools/check-style.sh`, which checks every project Zig file for:

1. tabs,
2. lines over 100 columns,
3. functions over 70 lines.

Formatting is necessary but not sufficient. `zig fmt` cannot prove that a name is precise or that a comment explains the correct invariant. Human review uses the checklist below.

52 Review checklist

Review a protocol change in this order.

1. State the invariant affected by the change.
2. Identify the durable write that preserves it across restart.
3. Confirm that no dependent message can leave before that write is synced.
4. Check duplicates, reordering, message loss, and a stale ballot.
5. Check one voter, a minimum quorum, and the configured maximum.
6. Confirm that all loops, buffers, and retries are bounded.
7. Read names and branches without relying on the author explanation.
8. Add a deterministic failure schedule test.
9. Run formatting, tests, examples, benchmarks, and the book build.
10. Make only the performance claim supported by the recorded measurement.

A formatter cannot grant clarity A file can be perfectly formatted and still be difficult to understand. Readability counts only when the state transition and its reason are visible.

PART V

Three worked systems

We now build a small counter, a useful key value service, and a large regional control plane. Each example keeps the same protocol and adds one engineering layer at a time.

53 Small example: one counter

The smallest useful state machine holds one integer. A command either does nothing or adds an amount.

examples/counter.zig

```
const Command = struct {
    client: u32,
    request: u32,
    operation: enum { noop, add },
    amount: i64,
};

const Consensus = paxos.Protocol(Command, .{
    .max_members = 3,
    .max_slots = 32,
});
```

Every node starts with the same membership.

```
var membership: Consensus.Membership = undefined;
try membership.init(&.{ 1, 2, 3 });

var nodes: [3]Consensus.Node = undefined;
try nodes[0].init(1, &membership);
try nodes[1].init(2, &membership);
try nodes[2].init(3, &membership);
```

The example uses an in memory queue. It campaigns node 1 with a no op, routes messages until the queue is empty, then proposes three additions of 10.

The state transition is almost too small to name.

```
for (effects.committedSlice()) |entry| {
    if (entry.value.operation == .add) {
        counters[node_index] += entry.value.amount;
    }
}
```

The result is:

```
proposed request 1 in slot 1
proposed request 2 in slot 2
proposed request 3 in slot 3
replicated counters: 30, 30, 30
```

53.1 Trace the first addition

Step	Actor	Event and reason
1	Client	Creates request `(7, 1, add 10)`.
2	N1	Assigns slot 1 and persists its local acceptance.
3	N1	Sends accept to N2 and N3.
4	N2	Persists and acknowledges. A quorum now exists.
5	N1	Records commit and sends commit to both peers.
6	All	Apply slot 1. Each counter becomes 10.

If N3 is stopped, steps through N2 still form a quorum. When N3 returns, it asks for commits from slot 1 and applies the same prefix.

53.2 What this example omits

The in memory disk cannot survive a process exit. The queue has no checksum or authentication. The client does not retry. Election is chosen by the program. The example teaches the library boundary, not a deployment.

Exercise 21.1. Drop every message to N3. Run the three additions. Restore communication and call `requestCatchUp(1, 1, &effects)` on N3. Predict the commit order.

54 Middle example: a key value service

We now store named byte values. The important new problem is client ambiguity. A client may send `put`, lose the reply, and retry. Consensus can place both copies in different slots. The state machine must make the logical request idempotent.

54.1 Command and state

```
const Command = struct {
    client_id: u128,
    request_id: u64,
    operation: enum { noop, put, remove, read_barrier },
    key: [64]u8,
    key_length: u8,
    value_hash: [32]u8,
};

const ClientRecord = struct {
    request_id: u64,
    result: Result,
};

const State = struct {
    values: BoundedMap([64]u8, [32]u8),
    clients: BoundedMap(u128, ClientRecord),
    applied_slot: paxos.Slot,
};
```

Large value bytes live in a content addressed blob store. Consensus orders a 32 byte hash. Before proposing, the leader makes sure the blob is durable on enough storage nodes. Applying the command only updates the small hash map.

54.2 Deterministic apply

```
fn apply(state: *State, slot: paxos.Slot, command: Command) !Result {
    std.debug.assert(slot == state.applied_slot + 1);

    if (state.clients.get(command.client_id)) |record| {
        if (command.request_id <= record.request_id) return record.result;
    }

    const result = switch (command.operation) {
        .noop => Result.noop,
        .put => try state.values.put(command.key, command.value_hash),
        .remove => state.values.remove(command.key),
        .read_barrier => Result.barrier,
    };
    try state.clients.put(command.client_id, .{
        .request_id = command.request_id,
        .result = result,
    });
    state.applied_slot = slot;
    return result;
}
```

Real code must define what happens when a client sends request 9 after request 7 but request 8 is missing. One policy permits monotonic request IDs and treats 9 as superseding 8. Another stores a bounded window. The policy is application semantics, not Paxos.

54.3 Write request

The server path is:

1. Authenticate the client.
2. Validate key and value bounds.
3. Store the value blob and obtain its hash.
4. Construct a command with a stable request ID.
5. Forward to the known leader or propose locally.
6. Persist protocol writes before sending protocol messages.
7. Wait until the assigned slot is committed and applied.
8. Return the cached state machine result.

A connection close at any point after proposal is ambiguous. The client retries the same `(client_id, request_id)`.

54.4 Linearizable read

The simple method proposes `read_barrier`. When that command is applied, every earlier chosen command has been applied. The server then reads its local map.

```
const barrier_slot = try node.propose(.{
    .client_id = client_id,
    .request_id = request_id,
    .operation = .read_barrier,
    .key = [_]u8{0} ** 64,
    .key_length = 0,
    .value_hash = [_]u8{0} ** 32,
}, &effects);
```

This method is not the fastest. It is easy to prove. Optimize reads only after the required consistency is written down.

54.5 Durable journal owner

One task owns the Paxos node. It receives network and client events through bounded queues. For each event it calls one node method and obtains effects.

```
event owner -> node.step
node        -> durable write batch
disk        -> durable completion
event owner -> transport sends
event owner -> state machine applies contiguous commits
```

No other task sends Paxos messages. This single owner makes the ordering rule easy to inspect.

54.6 Snapshot

Assume the log bound is 100,000 slots. At slot 80,000 the leader proposes a checkpoint command. Once applied, each node writes:

```
snapshot_version
cluster_epoch
applied_slot = 80000
key_value_state
client_result_table
checksum
```

The snapshot is complete only after its checksum and final marker are durable. A new epoch can then begin under an application barrier. The old journal remains available until the new epoch has a verified quorum.

54.7 Failure story

N1 commits a put in slot 401 but crashes before replying. N2 becomes leader. Its phase one recovery sees the accepted value and reproposes it. The client retries the same request. N2 may put that retry in slot 402. When the state machine applies 402, the client table returns the result from 401 without storing the blob twice.

Paxos prevents two values in slot 401. The client table prevents one logical request in two slots from running twice. Both layers are necessary.

55 Large example: a regional control plane

Consider a service that assigns workloads to regions. It manages 50,000 workers and receives 20,000 commands per second at peak. A wrong order can assign one worker twice. A stale read is acceptable for dashboards but not for placement.
We choose five voting nodes across three failure domains.

Node	Region	Zone	Duty
`N1`	`east`	`east-a`	Preferred leader.
`N2`	`east`	`east-b`	Nearby voter.
`N3`	`central`	`central-a`	Voter and snapshot source.
`N4`	`west`	`west-a`	Remote voter.
`N5`	`west`	`west-b`	Remote voter.

A quorum is three. The layout tolerates any two node failures, but it does not survive loss of east plus one west node if only central and one west remain. The failure domain calculation must use actual placement, not just `N = 5`.

55.1 Command design

```
const Command = struct {
    tenant_id: u64,
    client_id: u128,
    request_id: u64,
    timestamp_ms: u64,
    kind: enum {
        noop,
        assign,
        release,
        change_quota,
        checkpoint,
        read_barrier,
    },
    payload_hash: [32]u8,
};
```

The timestamp is data chosen before proposal. Apply never reads a local clock. The payload contains worker sets and constraints in external durable storage. Its hash enters consensus.

55.2 Sharding

One Paxos group cannot order an unlimited world. We partition tenants among 64 groups. Each group has its own membership, node state, log, applied state, and client deduplication table.
The shard map is itself versioned configuration. A command carries the shard map version used by the client gateway. A stale version is rejected or routed through a controlled migration. Moving a tenant between groups is a distributed transaction and is not made atomic by either group alone.

55.3 Capacity sketch

At 20,000 commands per second, an epoch with 10 million slots lasts:

```
10,000,000 / 20,000 = 500 seconds
```

That is too short. Batching 100 application commands into one consensus value reduces the slot rate to 200 per second:

```
10,000,000 / 200 = 50,000 seconds, about 13.9 hours
```

Still short for comfortable operation. A bounded in memory Paxos log is not the right place for months of history. The control plane should use smaller epochs, regular snapshots, and an external audit log.

The batch value contains a fixed maximum number of command hashes. The leader flushes when the batch is full or a short latency timer expires. The timer decides when to propose. It is not read during deterministic apply.

55.4 Network sketch

If one batch is 8 KiB and the leader sends it to four peers at 200 batches per second, leader egress for accept payloads is roughly:

```
8 KiB * 4 * 200 = 6.25 MiB/s
```

Commit messages are small. A leader change may send recovery metadata for many slots, so snapshots and epoch length must also bound recovery time.

55.5 Disk sketch

One fsync per batch means 200 syncs per second. A device with 2 ms median sync latency can keep up in the average case, but tail latency matters. Grouping local accept records and journal metadata into one atomic write avoids extra syncs.

The benchmark in this repository uses memory storage. It says nothing about these disk numbers. A deployment benchmark must include the actual journal.

55.6 Read classes

Caller	Read path	Reason
Dashboard	Local follower read with applied slot.	Staleness is visible and acceptable.
Scheduler	Log barrier through the leader.	Placement needs linear order.
Audit export	Snapshot plus committed suffix.	Bulk work need not block the proposal path.

55.7 Observability

Every group exports:

1. current role, ballot, and leader hint,
2. highest committed and applied slot,
3. proposal queue depth and age,
4. journal append and sync latency,
5. messages by tag and rejection reason,
6. campaign count and leader tenure,
7. catch up distance per peer,
8. remaining slot capacity,
9. snapshot age and checksum status.

An alert on remaining slots is a safety feature. An alert after exhaustion is a postmortem note.

55.8 Regional failure

Suppose the east region loses N1 and N2. N3, N4, and N5 can form a quorum. The failure detector eventually chooses one candidate. Its phase one reads a complete quorum and recovers every constrained slot. Client gateways redirect new work.

Cross region latency now defines commit latency. Safety is unchanged. Capacity may fall because the surviving leader has different network and disk limits.

When east returns, its old leader messages carry lower ballots and are rejected. The nodes catch up before serving strong reads. Merely reconnecting a process does not make its state current.

55.9 Security boundary

The large system authenticates peers with mutually authenticated channels. The transport binds the certificate identity to the Paxos node ID and cluster epoch. It rejects an envelope whose claimed source differs from the channel identity.

Payload hashes are verified before apply. Frame length is checked before memory is reserved. Rate limits apply before expensive signature or blob work when possible. Paxos is not a substitute for peer authentication.

55.10 What remains outside this library

The example needs automated epoch transition, snapshot transfer, batch values, read index optimization, durable journal framing, and a failure detector. Those features belong to the system design. The current library supplies the bounded ordering core and states its limits rather than offering incomplete switches.

PART VI

Evidence

We inspect the proof with tests, inspect the implementation with measurements, and inspect a deployment with failure drills.

56 Testing a consensus core

A normal test proves that one schedule works. A consensus test should search for schedules that almost break the invariant.

The main safety oracle is simple:

```
for every slot:
    collect every committed value from every node
    assert that the set contains at most one distinct value
```

The liveness oracle is conditional:

```
after the network becomes stable and one leader remains:
    every accepted client command eventually commits
```

Do not assert liveness while messages are dropped forever. The protocol cannot manufacture a quorum.

56.1 Deterministic schedule tests

The repository tests these cases:

Test	Property
Ballot order	Rounds dominate nodes and equal rounds use node order.
Membership errors	Empty, zero, and duplicate IDs are rejected.
Consecutive commit	Three nodes learn the same two values.
Duplicate messages	Prepare and accept repeats do not add writes or votes.
Reordered recovery	Completion markers may precede entries.
Highest vote	Recovery chooses the greatest accepted ballot.
Local acceptance	The leader persists locally before remote messages.
Contiguous learning	Slot 2 waits for slot 1.
Catch up	Known commits are returned from the requested slot.
Slot exhaustion	The final slot is not reused.

56.2 A simulator

A stronger simulator keeps a bounded queue of envelopes. At each step it chooses one action from a seeded generator:

1. deliver one envelope,
2. drop one envelope,
3. duplicate one envelope,
4. move one envelope to another queue position,
5. campaign one node,
6. crash one node,
7. restore one node from its durable shadow,
8. heal a partition,
9. propose one command.

After every action it checks safety. It also compares each live node's internal durable state with the state obtained by replaying the writes that the simulated disk accepted.

Seeds must be printed on failure. A failing schedule should be reduced to the smallest sequence that still fails. Random testing without reproducibility is a demonstration, not a debugging tool.

56.3 Crash injection

The effect boundary gives exact crash points.

Point	Simulation action
Before writes	Discard the whole effect batch and restore.
During writes	Apply a prefix or simulate a torn record, according to the journal contract.
After sync	Apply all writes, discard messages, and restore.
During sends	Apply all writes and enqueue only a message prefix.
During commit apply	Persist an application prefix and test at least once delivery after restore.

56.4 Model checking

The code is not a formal specification. A small model can still mirror its core state: promises, accepted ballots, accepted values, and chosen values for two slots and three nodes. Exhaustive exploration can check that no transition makes two chosen values.

The model and implementation must be compared field by field. A proof of the wrong model is no proof of the program.

57 The cross language benchmark

The benchmark uses two independent implementations. OmniPaxos 0.2.2 is a Rust replicated log library from crates.io. LibPaxos3 is a C implementation from the University of Lugano. Its source is pinned at revision `d255f8b`.
The command is:

```
zig build benchmark
```

It runs the Zig executable in `ReleaseFast`. It then runs the Rust package with release optimization, one code generation unit, and fat link time optimization. Finally it compiles the unmodified LibPaxos3 core with `zig cc -O3 -DNDEBUG`. `Cargo.lock` pins all Rust dependencies. The C script verifies its complete Git revision before compilation.

57.1 Workload

Item	Value
Voting nodes	3
Values per sample	4,096 sequential `u64` values.
Samples	7. The median elapsed sample is reported.
Storage	In memory for all implementations.
Network	In process delivery for all implementations.
Leader setup	Completed before timing and message counting.
Completion	The queue drains after every proposed value.
Correctness	Every checksum must equal 8,390,656.

The benchmark is a latency shaped microbenchmark. It does not batch many client values into one consensus message. It does not serialize, call a kernel network, or sync a disk. Zig and Rust use a stable leader and send six messages per value. LibPaxos3 maintains its 128 slot phase one preexecution window and sends twelve.

57.2 One observed run

On July 18, 2026, one arm64 macOS 26.5.1 run with Zig 0.16.0 and Rust 1.95.0 reported:

Implementation	ns/value	messages/value	checksum
Zig Multi Paxos	`206.41`	`6.00`	`8390656`
C LibPaxos3	`1679.44`	`12.00`	`8390656`
Rust OmniPaxos	`4524.64`	`6.00`	`8390656`

The first honest comparison found that the Zig code sent 9 messages per value because the leader sent messages to itself. The implementation was corrected to persist local acceptance directly and send only to peers.
The next comparison found a more important error. The benchmark constructed a large bounded `Effects` object inside the drain function for every value. The sampling profiler showed almost all time in `bzero`. Reusing one caller owned effect buffer changed the observed Zig result from about 11,588 ns per value to between 113 and 206 ns in subsequent runs. The protocol did not become one hundred times faster. The benchmark stopped measuring unnecessary memory clearing. Since the timed Zig sample is below one millisecond, normal scheduling noise remains visible. Treat the result as a regression signal.

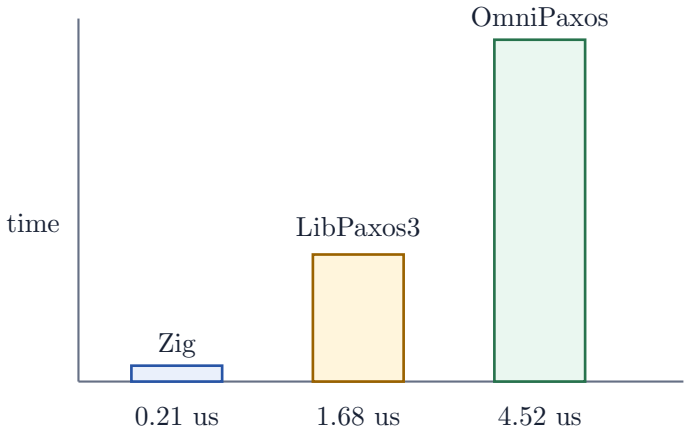


Figure 8: Observed local CPU time per value. This is a regression signal, not a universal ranking. Lower is better.

In this observed CPU measurement, Zig is about 8.1 times faster than the C run and about 21.9 times faster than the Rust run. These are local ratios for this workload. The C implementation performs twice as many logical messages and uses a different phase one design. The libraries also have different features and data structures. The numbers must not be presented as a universal language ranking.

57.3 What the benchmark does not prove

It does not prove which library is faster on another CPU. It does not measure durable latency. It does not compare recovery, batching, reconfiguration, snapshots, read paths, serialization, or partial connectivity. The feature map in `docs/features.md` records relevant differences.

For a publishable performance claim, record:

1. exact source revisions and dependency lock files,
2. CPU model, core pinning, governor, memory, and operating system,
3. compiler versions and flags,
4. warmup, sample count, distribution, and confidence interval,
5. payload sizes and batching,
6. storage and network configuration,
7. median and tail latency as well as throughput.

58 Performance lessons from C and the profiler

LibPaxos2 uses a fixed circular instance table, a compact promise bit vector, phase one preexecution, and buffered UDP messages. LibPaxos3 separates its core from libevent, maintains a preexecution window, and tracks a quorum with a member array and count. MySQL XCom also uses bit sets for prepare and propose replies.

The Zig core already had contiguous fixed arrays and a stable leader pipeline. This revision added exact width bit sets for duplicate promise and vote tracking. It also made effect buffer initialization explicit and reused that buffer in the example and benchmark.

Further work should be measured.

1. Benchmark several payload sizes and batch sizes.
2. Measure wire encoding and packet batching.
3. Measure a durable journal with realistic sync policy.
4. Separate phase one output capacity from steady phase two output capacity if large bounds make effect buffers impractical.
5. Profile before changing branch structure.

Any optimization must retain the durable write order and deterministic tests. A faster unsafe acknowledgement is not an optimization.

59 Operating drills

Before production, rehearse:

1. one follower crash and restore,
2. leader crash before any accept reply,
3. leader crash after a quorum but before commit broadcast,
4. minority partition,
5. delayed old leader traffic after healing,
6. disk full during promise and accept writes,
7. torn final journal record,
8. snapshot transfer interrupted at every chunk,
9. slot capacity alert and epoch transition,
10. loss of a complete failure domain.

For each drill, record the expected role, ballot, commit prefix, applied prefix, client response, and recovery time. If the expected result cannot be written before the drill, the design is not yet understood.

60 Review checklist

Area	Question
Identity	Can a node ID ever name another logical member?
Durability	Can any protocol reply leave before its write is synced?
Decode	Are length and version checked before expensive work?
Application	Is apply deterministic and ordered by slot?
Retry	Can one client request appear in two slots without running twice?
Read	Does each endpoint state its consistency level?
Election	Will campaigns eventually stop competing?
Snapshot	Is applied slot atomic with application state?
Capacity	Is transition tested before the hard slot bound?
Security	Does authenticated identity match the envelope source?

PART VII

Desk reference

This part collects messages, state, errors, formulas, and answers in one place.

61 Message reference

Message	Fields	Meaning
`prepare`	`ballot, decided_through`	Ask an acceptor to promise this ballot and report accepted state above `decided_through`.
`promise`	`ballot, slot, accepted`	Report one accepted slot for phase one recovery.
`promise_done`	`ballot, accepted_count, decided_through`	State how many distinct promise entries complete this reply and report peer's decided through index.
`accept`	`ballot, slot, value`	Ask an acceptor to vote.
`accepted`	`ballot, slot, decided_through`	Acknowledge a durable vote and report peer's decided through index.
`commit`	`slot, value`	Teach a chosen value to a learner.
`learn`	`from_slot`	Ask a peer for known commits.
`nack`	`rejected, promised, decided_through`	Reject work below a durable promise and report peer's decided through index.
`heartbeat`	`ballot, decided_through`	Leader heartbeat reporting its decided through index.

62 Durable writes

Write	Fields	Required before
`promise`	`ballot`	Promise replies for that ballot.
`accept`	`ballot, slot, value`	Accepted acknowledgement.
`commit`	`slot, value`	Application delivery and catch up claim.

63 Node state

Field	Meaning	Lifetime
`durable.promised`	Highest ballot this acceptor permits.	Durable
`durable.accepted`	Last accepted ballot and value per slot.	Durable
`durable.committed`	Known chosen value per slot.	Durable
`role`	Follower, preparing, or leader.	Volatile
`ballot`	Current local attempt.	Volatile
`highest_observed_round`	Round learned from higher traffic.	Volatile
`next_slot`	Next free proposal slot or zero.	Rebuilt
`delivered_through`	Prefix emitted in this process.	Volatile
`promise_*`	Phase one receipt accounting.	Volatile
`recovered`	Greatest accepted report per slot.	Volatile
`proposals`	Leader value per active slot.	Volatile
`acknowledgements`	Member votes per slot.	Volatile

64 Error reference

Error	Meaning
`EmptyMembership`	No voter was supplied.
`TooManyMembers`	Membership exceeds `max_members`.
`InvalidNodeId`	Node zero was supplied.
`DuplicateNodeId`	One identity occurs twice.
`NotMember`	A local or source identity is outside membership.
`WrongRecipient`	The envelope target is not this node.
`NotLeader`	Proposal was attempted before completed phase one.
`BallotExhausted`	No greater `u64` round exists.
`InvalidSlot`	Slot zero was supplied where a real slot is required.
`SlotLimitReached`	The bounded log has no free slot.
`InvalidPromise`	A reply count exceeds the slot bound.
`MissingNoop`	Recovery needs a no op but none was supplied.
`ConflictingValue`	One ballot and slot carried two values.
`ConflictingCommit`	One slot was taught two committed values.
`PromiseRegression`	Journal replay tried to move a promise backward.

65 Formula sheet

Quantity	Formula	Example
Uniform majority quorum	`floor(N / 2) + 1`	`N=5` gives `3`.
Crash tolerance	`N - quorum`	`5 - 3` gives `2`.
Stable messages, this three node path	`2(N - 1) + (N - 1)`	`2 + 2 + 2 = 6`.
Epoch duration	`slots / slots_per_second`	`10M / 200 = 50,000 s`.
Accept payload egress	`payload * peers * rate`	`8 KiB * 4 * 200`.

66 Invariants for review

1. Ballot pairs are unique and totally ordered.
2. Every quorum intersects every quorum in the same membership epoch.
3. An acceptor never accepts below its durable promise.
4. One ballot and slot have at most one value.
5. A new leader uses the value from the greatest accepted ballot reported by a complete phase one quorum.
6. A value is called chosen only after a quorum accepts it.
7. One slot has at most one committed value.
8. Application entries are released only as a contiguous slot prefix.
9. Durable writes precede every message that depends on them.
10. Slots are never reused within an epoch.

67 Answers to selected exercises

67.1 Exercise 1.1

A majority of five is three. Two nodes may be unavailable while three remain.

67.2 Exercise 2.1

`{A1, A2}` and `{A3, A4}` do not intersect. If both were quorums, they could choose different values without sharing a witness.

67.3 Exercise 4.1

Choose `apple` from ballot 9. Knowledge that ballot 3 succeeded does not change the procedure. The induction says the later legal vote at ballot 9 must already preserve any earlier chosen value.

67.4 Exercise 8.1

The promise write precedes the promise reply. Each accept write precedes its accepted reply. The commit write precedes local application and later catch up.

68 Glossary

Term	Meaning
Accepted	One acceptor durably voted for a ballot, slot, and value.
Applied	The application state machine executed a committed entry.
Ballot	A unique ordered proposal attempt.
Chosen	A quorum accepted one value in one slot.
Commit	A learner's record of a chosen value.
Epoch	One period with fixed membership and nonreused slots.
Leader	A candidate that completed phase one for its ballot.
Learner	A role that discovers chosen values.
No op	A valid application value that changes no state.
Promise	An acceptor's durable refusal to accept lower ballots.
Quorum	A voter set that intersects every other legal quorum.
Slot	A one based position in the replicated log.

69 Further study

Start with the local paper `docs/lamport-paxos.pdf`. Then read Lamport's "Paxos Made Simple". Compare the state with ZooKeeper's Zab recovery and with the plain struct boundary of OmniPaxos. Read TigerStyle beside the Zig source. Its rules about bounds, assertions, initialization, and comments are especially relevant to consensus code. LibPaxos is useful for studying a small C core, preexecuted phase one windows, buffered transport, and the cost of heap based quorum state. MySQL XCom shows a larger production C and C++ design with compact reply sets.
Useful links:

- https://lamport.azurewebsites.net/pubs/paxos-simple.pdf
- https://cwiki.apache.org/confluence/display/ZOOKEEPER/Zab1.0
- https://omnipaxos.com/
- https://bitbucket.org/sciascid/libpaxos
- https://dev.mysql.com/doc/dev/mysql-server/9.7.0/xcom___base_8h_source.html
- https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/TIGER_STYLE.md
- https://typst.app/universe/package/fletcher
- https://typst.app/universe/package/cetz

70 Closing note

Paxos is not a bag of messages. It is one rule carried through time. A later ballot must respect what an earlier quorum may have chosen. Quorum intersection finds a witness. Stable storage lets the witness remember. Phase one asks the witness. Phase two records the next fact. Slots put facts in order.

The rest is engineering. That phrase does not mean the rest is easy. It means the proof has given the engineering a fixed point. Every queue, disk write, retry, snapshot, and benchmark can now be judged by whether it preserves that point.